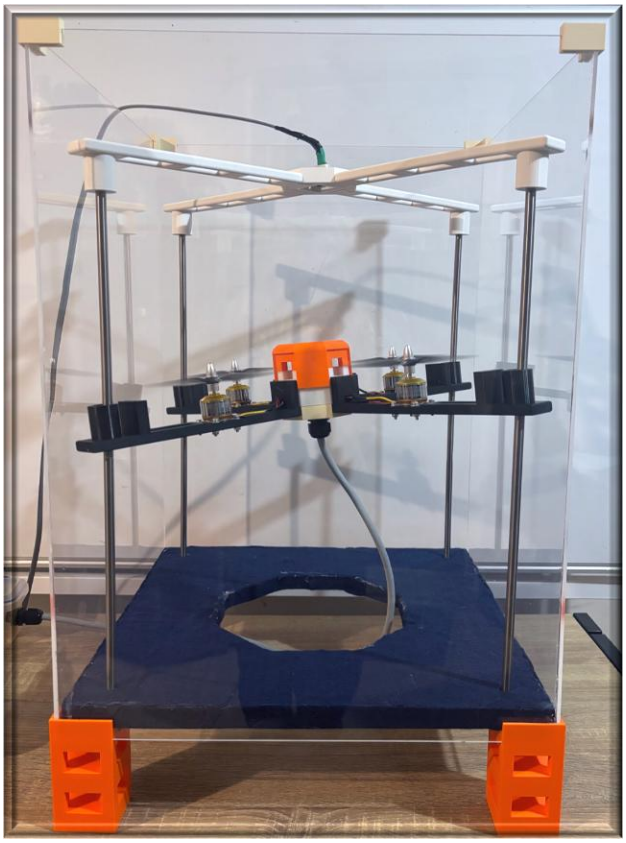
2025



Developing the PID-Controlled Flight Rig for use in ENEL321, 2026

CHANELLE DUFFY

Abstract

This project refines the PID Flight Platform developed in ENEL300 into a reliable and demonstrative teaching tool for the ENEL321 Control Systems course in 2026. Improvements include a live data acquisition and visualisation interface that plots platform height and calculates key performance metrics such as rise time, settling time, overshoot, and steady-state value in near real time. A mathematical model of the system was developed to simulate proportional and proportional-derivative controller responses, incorporating disturbances such as cable spring effects and ground effect to improve accuracy. Noise testing confirmed that variations were primarily caused by disturbances rather than sensor or actuator noise. Experimental results show strong agreement with the model, with the proportional controller achieving over 90% accuracy in rise time and steady-state values. The enhanced platform provides an interactive and reusable tool for visualising PID control behaviour, supporting hands-on exploration of controller tuning and bridging the gap between theory and practice.

CONTENTS

1. Introduction

Control systems are a fundamental part of modern engineering, appearing across all areas of autonomous technology. At the University of Canterbury, students are formally introduced to control systems in their third year, where the subject is taught largely at a conceptual and theoretical level. In the previous ENEL300 Design and Build project, an initial PID Flight Platform was developed to bridge the gap between theory and practice by providing a tangible demonstration of control system dynamics. The prototype successfully demonstrated closed-loop control for a single-axis drone platform, and laid the groundwork for use as a teaching aid in the ENEL321 Control Systems course.

Building upon this foundation, the focus of this project was to refine and enhance the prototype to make it suitable for use in the 2026 ENEL321 course. Key areas of improvement included implementing real-time data visualisation, developing a mathematical model of the system to enable accurate simulation and analysis, and conducting noise and disturbance analysis to better understand system behaviour. These developments were aimed at improving both the educational value and reliability of the platform.

This report outlines the design, development, and evaluation of the enhanced PID Flight Platform. It is structured as follows:

- **Context and Requirements** - background, purpose, and key stakeholder needs.
- **Design** – Overview of the main problems addressed – noise analysis, control model, live graphing and key metric calculations.
- **Evaluation –** Analysis of experimental data and assessment against requirements.
- **Remaining Tasks –** Identified improvements and next steps.
- **Sustainability –** Analysis against sustainable practices.
- **Conclusion**

By improving system accuracy, responsiveness, and analytical capabilities, this project strengthens the educational impact of the original design, turning the PID Flight Platform into a reliable and interactive teaching tool that makes abstract control concepts visible and engaging for students.

2. Context

2.1 Background

Control systems are fundamental to electrical and electronic engineering, underpinning virtually all modern autonomous and automated technologies. They enable systems to sense, decide, and act in response to changes in their environment through continuous feedback loops. Among the various control strategies, Proportional–Integral–Derivative (PID) control remains the most widely used in industry due to its simplicity, reliability, and effectiveness across a broad range of applications [1].

In the University of Canterbury's ENEL321: Control Systems course, PID control is a core topic taught by Christopher Hann, yet there has historically been a lack of a physical demonstration tool that allows students to clearly observe how the theoretical concepts translate into real-world behaviour. Earlier this semester an initial teaching aid prototype was developed during the ENEL300 Design and Build project to address this gap. The system successfully demonstrated basic closed-loop control on a single-axis hover platform using a LiDAR sensor and an Arduino-based PID controller.

While the prototype proved valuable as a teaching concept and was even demonstrated to first-year students, it had several technical limitations that reduced its reliability and effectiveness. The most significant of these were the imperfect control tuning, and the absence of real-time data visualisation to connect system behaviour with theoretical concepts.

This project builds directly upon that foundation, aiming to refine the system into a robust, user-friendly, demonstrative teaching tool suitable for use in ENEL321 in 2026. The focus is on improving the control system responsiveness, enhancing data visualisation through a live graphical interface, and analysing key performance metrics such as rise time, settling time, and overshoot to strengthen the link between theory and practice.

2.2 Key Stakeholder

This prototype was developed for use in the third-year ENEL321 Control Systems course to demonstrate PID control concepts to students. As the course lecturer, Christopher Hann is the primary stakeholder, and the prototype was designed to serve as a practical teaching aid under his supervision.

2.3 Problem Definition

Although the existing prototype successfully demonstrated basic control concepts, it lacked the responsiveness, visual clarity, and analytical features required for use as a reliable teaching aid. Specifically:

- There was no live data plotting, meaning students could not visualise control response dynamics in real time.
- Performance metrics such as rise time, settling time, and overshoot were not automatically calculated, preventing clear analysis.
- The control model had not been developed or simulated, restricting understanding of how the chosen gain values influenced system behaviour.

The problem this project addresses is how to further enhance the existing control systems prototype to make it ready to be used in the classroom in 2026. The objectives of this project are drawn from the next steps outlined at the end of ENEL300 project 1. These next steps were developed by both the key stakeholder Chrisopher Hann, and the project 1 team.

2.4 Objective/Requirements

The primary objective of this project is to develop the existing single-axis drone control demonstrator into a reliable and engaging teaching aid that can be used by Christopher Hann for the ENEL321 Control Systems course in 2026. The system should effectively bridge the gap between theoretical control concepts and their real-world implementation by allowing students to both observe and analyse the dynamic behaviour of a PID-controlled platform.
To achieve this, the following requirements were defined in table 1.

Table 1: Project Requirements

No.	Requirement	Acceptance Criteria	Rationale	Priority
1	**Control Model**			
1.1	Derive an equation of motion of the system.	Yes/No	Provides a baseline for the system identification.	Essential
1.2	At least one major disturbance that significantly affects system behaviour must be included in the control model.	Yes/No	If disturbances are playing a major role in the system, than any model excluding them will not be an accurate representation for tuning purposes.	Essential
1.3	Simulated response is reasonably accurate to the experimental response.	Key metrics such as rise time and overshoot of the simulated model is > 60% accurate to the same metrics of the experimental data.	An accurate model is essential for tuning gains for the system.	Desirable
2	**Data Acquisition and Visualisation**			
2.1	A live viewer must plot the height measurement data at a frequency of ≥ 20 Hz.	Yes/No	This is the minimum update frequency that resembles live plotting, any less will have too much of a delay.	Essential
2.2	The data from previous flights (plots and performance metrics) should be retained up until the program is reset.	Yes/No - Picture Evidence	Allows for analysis post demonstration.	Desirable
2.3	Previous flight plots can be replayed post demonstration.	Yes/No - Video Evidence	Allows for analysis post demonstration.	Optional

2.4	The user can stop recording data through either a hardware or software interrupt.	Yes/No	To prevent the landing and rest data disrupting the metric calculations and graphical plots.	Essential
2.5	Plots from previous flights should be able to be viewed on the same screen.	Yes/No - Picture Evidence	To allow for comparison between different gains and controllers.	Optional
2.6	Provide a simple user interface to run the graphing program.	The user interface does not require the terminal to run the python script.	Improves user experience, makes it accessible to all audiences.	Optional
2.7	The plotting system should allow the appearance of the graph to be adjusted in code.	The line width, line colour, and background colour can be adjusted in python.	To ensure visibility in class demonstrations.	Desirable
2.8	The plotting system must integrate easily with Python. There should be valid documentation surrounding the plotting system and its integration with python.	Yes/No	Python has access to a number of libraries making it ideal for this purpose. The main library being Pyserial to allow for communication between Arduino IDE and python.	Essential
3	**Performance Metrics**			
3.1	Compute and displays rise time, steady state value and overshoot where applicable for each flight.	Yes/No - Picture Evidence	Provides further information for analysis post demonstration.	Essential
3.2	Compute and displays peak value, peak time and settling time where applicable for each flight.	Yes/No - Picture Evidence	Provides further information for analysis post demonstration.	Optional
3.3	The calculated steady state value is within ±10mm of the average of the visually determined steady state window.	Yes/No	The majority of the parameters depend on the calculation of the steady state value, the accuracy of this determines the accuracy of the rise time, overshoot, and settling time.	Essential

4	The live graph program and associated scripts are easily accessible on other computers.	Yes/No	Simplifies the setup required to perform demonstrations.	Optional

3. Noise Analysis

When the drone was hovering at its fixed gravity-offset duty cycle (43.5%), some variation in its movement was observed. This was expected, as several disturbances act on the system during operation. However, to develop an accurate control model, it was first necessary to determine whether sensor noise contributed significantly to these variations.

To confirm this, an experiment was conducted. This involved collecting measurement data while mechanically bracing the drone at its setpoint (210mm), while running at the fixed gravity offset PWM value. Once sufficient data had been gathered, the drone was released to hover freely at the same setpoint, and additional measurements were collected over time. These datasets are compared in Table 2 and Figure 1.

Table 2: Comparison between braced and unbraced data sets.

Braced Vs Unbraced	Braced	Unbraced
Average	219 mm	234 mm
Standard Deviation	1.12 mm	8.44 mm
Maximum Deviation	±3 mm	±18 mm

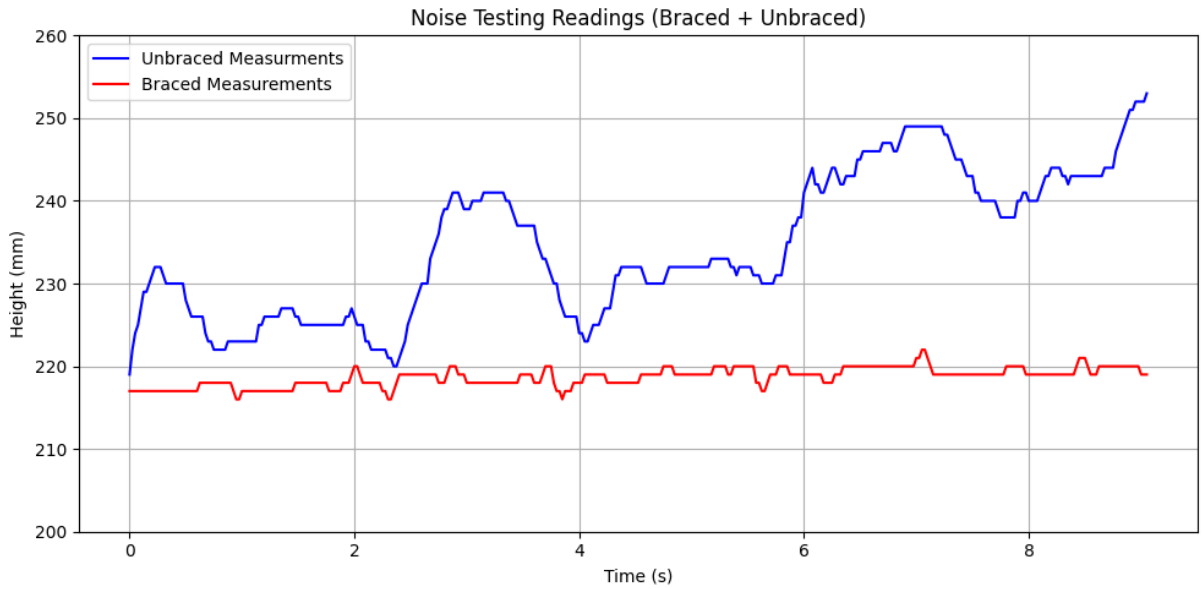


Figure 1: Noise comparison between braced and unbraced.

As shown in the table and figure, the braced data exhibits minimal deviation, demonstrating that sensor noise is negligible. Consequently, the majority of the observed variations during free hover can be attributed to external disturbances. This is the ideal outcome, as disturbances can be accounted for in the control model, whereas unpredictable sensor noise is much more challenging to model accurately.

4. Control Model

By developing a transfer function for the system, the response of the control model can be simulated. A transfer function for a proportional controller and a proportional-derivative controller was developed. The integral component was excluded from the model due to the added complexity.

The mathematical model was implemented in Matlab to allow for graphical plotting. The modelling was carried out by key stakeholder Chris Hann, he developed the original Matlab code for the proportional controller, which was then adapted for the PD controller. A free body diagram of the system can be seen below (Figure 2), the model accounts for everything in this diagram, including the disturbance introduced by the hanging power cable and ground effect.

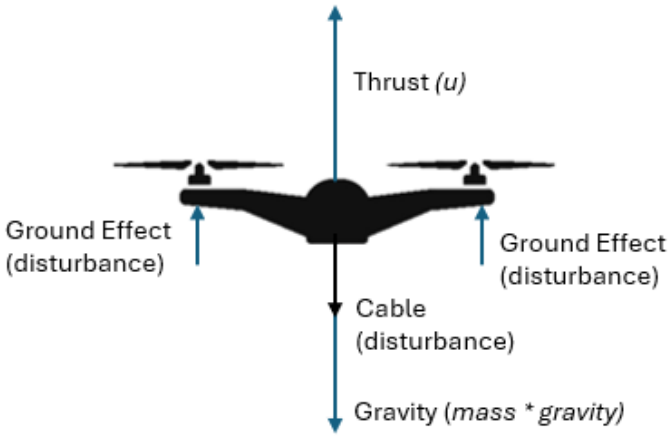


Figure 2: Free body diagram of simplified system.

4.1 Simulated Response

Due to the hardware constraints of the system, limiting it to one degree of freedom, the equation of motion could be simplified. The EOM was developed under the following assumptions:

- Air resistance was assumed negligible, as the primary aerodynamic influence on the system is the ground effect, which is modelled as a disturbance.
- Any forces acting outside of the one degree of freedom are considered negligible.

Under these assumptions the system could be simplified to solely the up and down forces (Equation 1).

Equation 1:Simplified Equation of Motion

$$m\ddot{z} = -mg + F_{Thrust} + d(t)$$

The disturbances are grouped into a disturbances function $d(t)$ for simplicity at this stage. To allow for comparison between the simulated model and experimental data, the gain values were set at the same value as the ones currently used in the flight rig. These being $K_p = 0.18$ and $K_d = 0.08$.

4.1.1 Thrust Coefficient

By simplifying Equation 1 to exclude the disturbances, and by modelling F_{Thrust} as a thrust coefficient (β) multiplied by the PWM value (u) and mass (m). This equation can be used to determine the relationship between the PWM value and the acceleration. At hover (u_0), acceleration is zero, using this the thrust coefficient can be derived (Equation 2), noting that the mass values have cancelled. The full derivation can be found in Appendix A.

Equation 2

$$\beta = g/u_0$$

The PWM percent value that is required to hover at the setpoint is 43.5, this is the gravity offset implemented in the original Arduino IDE code. Multiplying that by four scales it to the four motors, this value is considered u_0. With this number, the thrust coefficient is calculated in Equation 3.

Equation 3

$$\beta = \frac{9.81}{4 \times 43.5} = 0.0564$$

4.1.2 Disturbance – Cable

To model the disturbance of the cable, the spring effect was considered. Where C_b is the damping coefficient of the system due to the cable. Using the relationship between the proportional gain and the thrust coefficient, the natural frequency can be determined as shown in Equation 4.

Equation 4

$$\omega_n = \sqrt{\beta K_p}$$

Knowing the natural frequency, the damping coefficient of the cable can be estimated using the relationship in Equation 5.

Equation 5: Damping

$$C_b = 2\zeta\omega_n = 2\zeta\sqrt{\beta K_p}$$

Although the damping effect introduced by the cable is relatively small, it still has a measurable influence on system dynamics. Therefore, a damping ratio of approximately 0.15 was assumed, resulting in a spring damping coefficient of 1.

4.1.3 Disturbance – Ground Effect

Ground effect is the phenomenon where air pressure builds up beneath the drone's propellers when it operates close to the ground, caused by the downward airflow being restricted by the surface below. This results in the drone requiring less thrust than expected to maintain hover, as a cushion of air effectively supports part of its weight. To model this behaviour, a disturbance was introduced to the original hover offset ($u_0 = 174$) to represent the reduced thrust required to counteract gravity. In this model, a disturbance value of 0.4 was applied and multiplied by four to account for the four motors, producing a new effective hover offset as shown in Equation 6.

Equation 6: Hover offset with ground effect.

$$u_0 = (4 * 43.5) - (4 * 0.4) = 172.4$$

With this new hover value, the disturbance can be calculated by rearranging Equation 2. This rearrangement and final disturbance value can be seen in Equation 7, this is modelled as a constant offset.

Equation 7: Disturbance

$$d = \beta u_0 - g = (0.0564 \times 172.4) - 9.81 = -0.08664$$

4.1.4 Transfer Functions

The transfer functions of the system describe the relationship between the PWM input and the resulting height output. These were developed using the identified thrust coefficient, with the effects of the cable and ground-effect disturbances incorporated into the overall model. Separate transfer functions were derived for both the proportional (P) and proportional–derivative (PD) controllers to evaluate system behaviour under each configuration. The proportional transfer functions are presented below, while the PD transfer functions are provided in Appendix A.

The first transfer function derived takes the spring effect of the cable into account. This is modelled in Equation 8. The second incorporates the ground effect disturbance d as defined in Equation 7, this can be seen in Equation 9.

Equation 8: cable Disturbance

$$G_r = \frac{\beta K_p}{s^2 + C_b s + \beta K_p}$$

Equation 9: P Transfer Function

$$G_d = \frac{d}{s^2 + C_b s + \beta K_p}$$

With both transfer functions defined, the system response can be simulated. MATLAB's lsim() function was used to apply a specified step input into to each transfer function, producing the respective outputs Y_r and Y_d. These outputs are then summed to yield the total simulated response.

4.2 Model Validation

To validate this mathematical model, measurement data was taken from both a proportional controller and a proportional-derivative controller. Over 1000 measurements were taken for each, this allowed enough time for the system to reach steady state. Due to the position of the sensor at the top of the flight rig these measurements had to be inverted to make sense on the plots. This results in the setpoint changing from 210 mm to 267 mm (see section 5.1 for more information). By plotting this data against time on the same graph as the simulated response the two can be compared.

The proportional model can be seen in Figure 3 and the proportional-derivative controller is modelled in Figure 4. This is a reasonably accurate model as demonstrated in the two plots, the blue line of the simulation closely matches the red of the actual experimental data.

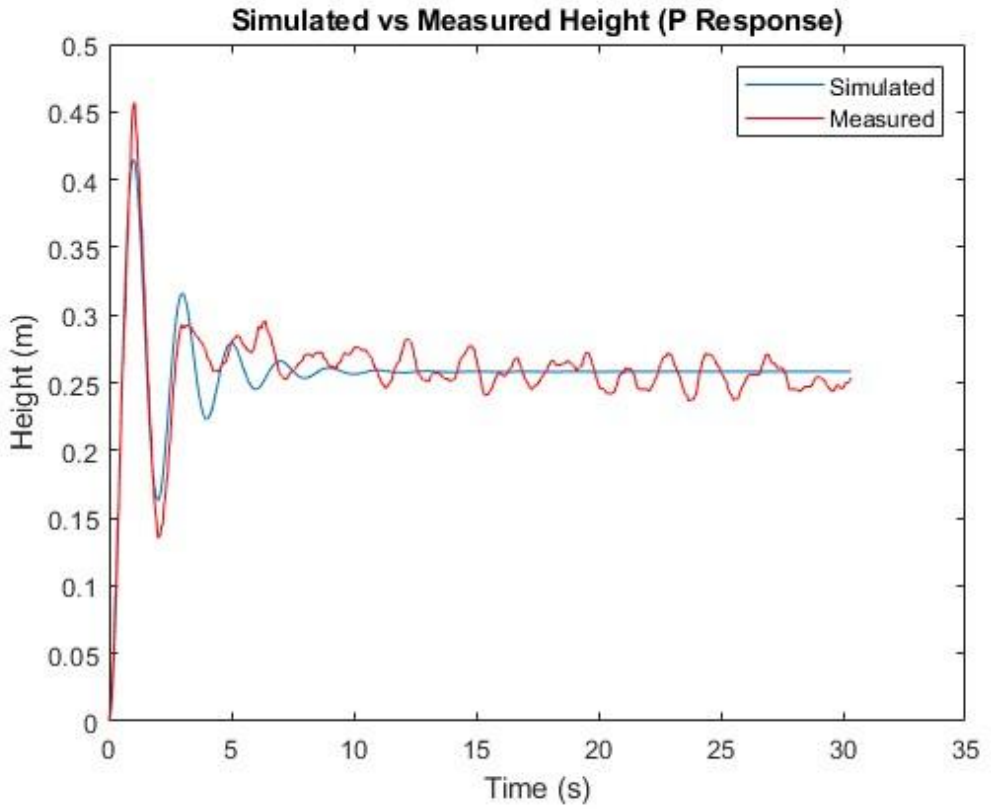


Figure 3: Proportional simulation vs experimental.

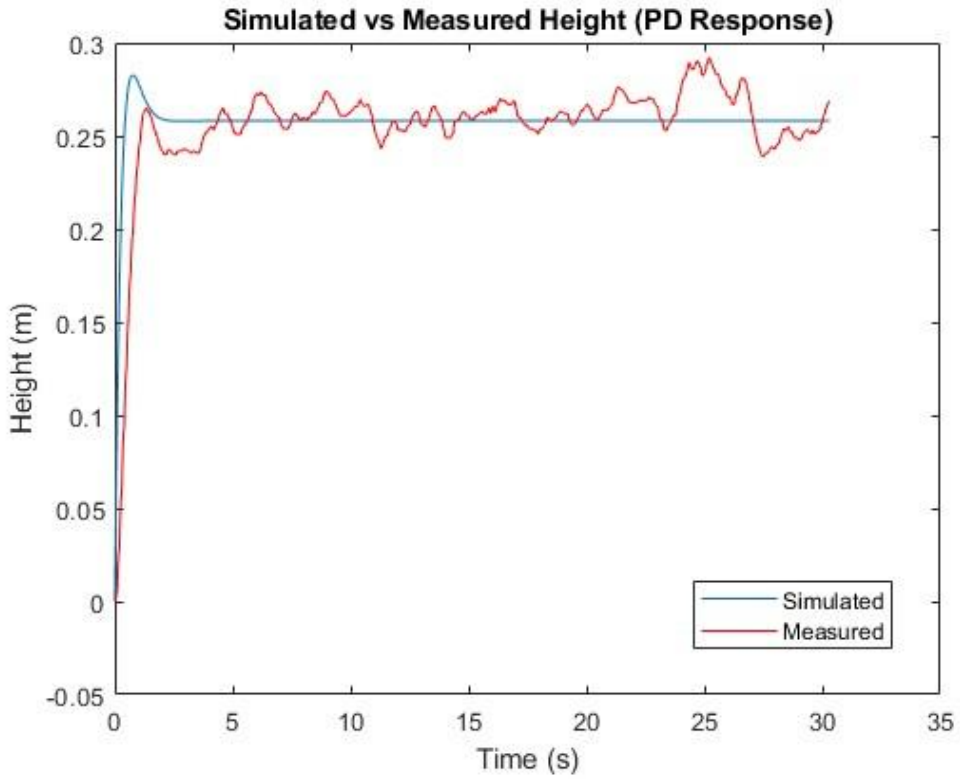


Figure 4: PD simulation vs experimental.

5. Data Acquisition and Visualisation

Data acquisition is a key component in enabling live graphical visualisation of the system's behaviour. The flight platform software was developed in the Arduino IDE to interface directly with the Arduino R3 board used in the design. However, this presented a challenge for real-time plotting, as most graphical interfaces do not integrate seamlessly with the Arduino environment. To overcome this, a Python script was implemented to serve as a bridge between the Arduino code and the live graphing interface. The complete Python script is available in the *ENEL300 Project 2* GitHub repository [2].

5.1 Measurement Data Handling

To pass the measurement data from Arduino IDE to python, the Pyserial library was utilised [3]. With this library, the data could be read from the serial port. In the Arduino IDE script, the measurement data from the sensor was printed to serial using the Serial.println(measurements) function in a single column format. This stream of data was read by python using the Pyserial library. The data was read from serial at a rate of 40Hz, this matches the control frequency and ensures near live plotting.

Due to the sensor being positioned at the top of the flight rig facing down, the measurement values had to be inverted. To do this,1000 readings were taken from the sensor with the flight platform in its rest position at the bottom. The mean of these readings was taken, giving a value of 472, this value was used as the offset to

invert the measurements. The large number of readings were necessary due to the variation in the sensor.

5.2 Live Graph Interface

To implement a live graphical feed within the time frame of this project, an external program was required. This program needed to do the following:

- Integrate easily with python.
- Be able to plot a live graph feed.
- Retain the data from the previous flights.

To avoid spending too much time on choosing a program, the experienced PhD student Narottam was consulted. He recommended the program Rerun, a free graphing software that integrates well with python. This program was investigated and found to meet all of the above requirements. Rerun also has plenty of documentation [4], making it simple to implement and debug.
The viewing side of rerun is structured in layers. These layers are wrapped in a blueprint that is updated at the beginning, and after each flight.

5.2.1 Blueprint Configuration

Rerun's blueprint is structured from the overarching view down to the specific graph type. Due to the requirement of being able to view previous flights, the first layer was set up in a tab format. This allows for simple switching between graphs. Within the graph view, there is a second layer setup for viewing both the plot and the summary text file.
Finally, the plots themselves are set up in the blueprint, with the graph being a time series plot and the text file as the textlog setting. The structure explained above can be seen implemented in python in Code Snippet 1.

```
def update_blueprint(origins, active_origin):
    """Build a master blueprint with one tab per flight."""
    tabs = []
    for o in origins:
        tabs.append(
            bp.Vertical(
                bp.TimeSeriesView(
                    origin=o,
                    axis_y=bp.ScalarAxis(range=(Y_AXIS_MIN, Y_AXIS_MAX), zoom_lock=True)
                ),
                bp.TextLogView(
                    origin=f"{o}/Summary_Metrics"
                ),
                row_shares=[7, 3],
                name=o.strip("/")
            )
        )
    tab_index = len(tabs) - 1
    blueprint = bp.Blueprint(
        bp.Tabs(
            *tabs,
            active_tab=tab_index,
            name="All Flights"  ) )
    rr.send_blueprint(blueprint)
```

Code Snippet 1: Blueprint update function.

5.2.2 Graph Reset

To start and end the flight of the drone, there is a red button on the user interface. This initiates the liftoff and landing sequences, as well as the printing to serial. Measurement data is only printed after the button has been pushed once (indicating the start of the flight). When the button is pressed again (indicating a landing) a special indicator is sent across serial in the form of the string “RESET”, and the measurement data is no longer sent. This is implemented to avoid plotting the data from the landing procedure. The python script looks for this RESET indicator and enters the graph reset function when it occurs.

This function handles the calling of the different calculation functions to print the key graph features, such as rise time and overshoot to the summary text file. These are added to the text file that is displayed beneath the graphical plot for each flight.

This function also handles the resetting of all arrays, and updating the blueprint file path to ensure a new tab is added if another flight is triggered.

5.3 Metric Summary

To allow for further analysis post demonstration, a metric summary text file is generated with every flight. This summary includes the following key metrics:

- **Peak Value** – The highest value reached (mm).
- **Peak Time** – The time it takes to reach the highest value (s).
- **Steady State Value** – The value the drone settles on (mm).
- **Overshoot** – The percent amount that the drone overshoots its steady state value (%).
- **Rise Time** – The time it takes to get from 10% to 90% of the steady state value (s).
- **Settling Time** – The time it takes to settle within a defined bound of its steady state value (s).

Excluding the steady-state value, these are all calculated post flight. This is due to most of the metrics reliance on the final steady-state value. As this platform was developed to demonstrate multiple different combinations of PID control, the system may not ever reach steady state. In this case, only the peak value and time is calculated, and the line “Never Reached Steady State” is printed to the summary file instead.

To perform these calculations post flight, two arrays are utilised, the first containing the height measurements (flight_data), and the second containing times (time_data). These lists are updated at the same time (see the limitations section for further information on latency), ensuring that every measurement has a corresponding time value.

5.3.1 Peak Calculations

These are the only calculations that do not rely on the steady state value. As such, they are also the simplest. These are performed in the peak_time_calc function seen below in Code Snippet 2.
The peak value is the maximum height measurement in the flight_data list. The peak time is therefore the value in the time_list that is at the same index as the peak value.

```
def peak_time_calc(flight_data, time_data):
    peak_value = max(flight_data)
    peak_index = flight_data.index(peak_value)
    peak_time = time_data[peak_index]
    return peak_time, peak_value
```

Code Snippet 2: Peak time/value helper function.

5.3.2 Steady State Calculation

To calculate the steady-state value, it was first necessary to determine whether the system had settled. This was achieved using a two-stage verification process based on a windowed moving average approach. The following checks are updated continuously during the flight, unlike the rest of the calculations which are conducted post flight.
The first stage used a defined SS_WINDOW value of 50 samples. Within this window, the maximum deviation of the samples was analysed to determine if it was less than the defined MAX_DEVIATION threshold. This threshold was derived from test data gathered under two different controller configurations and, for this system, was set at ±20 mm. A tighter bound can be applied if required.

The second stage was only executed if the first stage passed. It performed a consecutive mean comparison against the previous mean value. The mean of the most recent 20 samples (SS_AVERAGE) was computed to provide an up-to-date average, which was then compared with the previous mean to ensure the difference between them was within ±8 mm. To prevent bias caused by overlapping data, this average was only recalculated every 20th cycle, ensuring that consecutive means were derived from distinct sample sets.
If both conditions were satisfied, the average of the two means was taken, rounded to the nearest integer, and recorded as the steady-state value. These checks were executed continuously after the 50th sample to ensure sufficient measurement data had been collected.

5.3.3 Overshoot

Once the steady-state and peak values had been determined, the overshoot was simple to calculate. The percent overshoot was calculated using Equation 10.

Equation 10: Overshoot

$$M_p = \frac{peak\ value - steady\ state\ value}{steady\ state\ value} \times 100\%$$

5.3.4 Rise Time

The definition of rise time was based on the one taught in ENEL321, that being the time it takes to go from 10% of the steady-state value to 90%. To calculate rise time the two arrays, flight_data and time_data, were needed, along with the steady-state value.

To find the time at these key points, the values at 90% and 10% of the steady-state value had to be determined. These values are assigned to the variables high_val and low_val respectively in code snippet (3).

Due to the 40Hz sampling rate, these exact measurement values may not be present in the flight_data array, to account for this the numpy library's .argmax function was utilised [5]. This function found the first occurrence of any value that was greater than or equal to the exact 90% and 10% values. The indices of these values were used to find the corresponding time values in the time_data array. Finally, to determine rise time the difference between the 10% time and the 90% was taken.

```
def rise_time_calc(steady_state, flight_data, time_data):
    flight_arr = np.array(flight_data)
    high_val = round(0.9 * steady_state)
    low_val = round(0.1 * steady_state)
    ssh_index = np.argmax(flight_arr >= high_val)
    ssl_index = np.argmax(flight_arr >= low_val)
    low_time = time_data[ssl_index]
    high_time = time_data[ssh_index]
    return(high_time - low_time)
```

Code Snippet 3: Rise time calculation helper function.

5.3.5 Settling Time

To determine the settling time, the defined MAX_DEVIATION bound was used. An upper and lower bound were determined by adding and subtracting this max deviation from the steady-state value. The numpy library was again utilised to find the last instance of either of these values occurring, the corresponding time value was obtained from the time_data array. This was considered the settling time, the full code snippet can be seen in Code Snippet 4.

```
def settling_time_calc(flight_data, time_data, steady_state):
    lower_bound = steady_state - MAX_DEVIATION
    upper_bound = steady_state + MAX_DEVIATION
    flight_arr = np.array(flight_data)
    time_arr = np.array(time_data)
    # Boolean mask: True if out of tolerance
    out_of_tol = (flight_arr < lower_bound) | (flight_arr > upper_bound)
    if np.any(out_of_tol):
            # last index where it's out of tolerance
        last_out_idx = np.where(out_of_tol)[0][-1]
        return time_arr[last_out_idx]
    else:
        return 0.0
```

Code Snippet 4: Settling time calculation helper function.

6. Evaluation/ Limitations

This project has achieved all essential requirements that were outlined at the outset. Table 3 below summarises any requirements that remain partially or fully unmet. R1.3 was partially met as the proportional model was extremely accurate (demonstrated in section 6.1.1). However the proportional-derivative model did not meet the acceptance criteria. R2.3, R2.6 and R4 correspond to the user interface and packaging of the graphing program, these were both considered optional and are addressed in the future work section.

Table 3: Unmet Requirement Summary

No.	Requirement	Acceptance Criteria	Rationale	Priority
1.3	Simulated response is reasonably accurate to the experimental response.	Yes - for the proportional model, the PD model requires further work.	An accurate model is essential for tuning gains for the system.	Desirable
2.3	Previous flight plots can be replayed post demonstration.	No – Requires further code	Allows for analysis post demonstration.	Optional
2.6	Provide a simple user interface to run the graphing program.	No – Currently accessed through the terminal	Improves user experience, makes it accessible to all audiences.	Optional
4	The live graph program and associated scripts are easily accessible on other computers.	No	Simplifies the setup required to perform demonstrations.	Optional

6.1 Evaluation Against Key Requirements

In this section, the key project requirements are evaluated in detail to demonstrate how the objectives have been met. Evidence and analysis are provided for the most critical requirements, while the complete evaluation, including all requirements and acceptance criteria, is presented in Appendix B.

6.1.1 Control Model

R2.3 – Simulated response is reasonably accurate to the experimental response.

To confirm the accuracy of the control model, the following metrics were calculated for both the simulated model and the experimental data. These metrics were

compared to determine if the simulated model was within the required threshold. The metrics compared were:

- Rise time
- Overshoot
- Steady State Value

These were calculated for the proportional controller and the proportional-derivative controller. A limitation to this comparison is its reliance on the algorithm defined in section 6.3.2. This same algorithm was used to calculate the steady-state value for the experimental data. Therefore the metrics produced for the experimental data are only as good as this algorithm. The results for the proportional controller are displayed in Table 4, with the results for the proportional-derivative controller in Table 5.

Table 4: Comparison of simulated vs experimental metrics for a proportional controller.

Metrics	Simulated	Experimental	Accuracy
Steady State Value (mm)	258	248	96.1%
Rise Time (s)	0.375	0.375	100.0%
Overshoot (%)	60.7	84.2	72.1%

Table 5: Comparison of simulated vs experimental metrics for a PD controller.

Metrics	Simulated	Experimental	Accuracy
Steady State Value (mm)	258	252	97.7%
Rise Time (s)	0.325	0.75	43.3%
Overshoot (%)	9.51	15.9	59.8%

The data in the above tables demonstrate the accuracy of the proportional model. Both the rise time and the steady state values are greater than 90% accurate to the experimental. All metrics for the proportional model achieve R2.3, with all demonstrating accuracy to > 60%.
The proportional-derivative model was generally less accurate than the proportional model. The difference in rise time was as low as 43%, this can also be confirmed visually by comparing the two data plots in Figure 3. The transient region at the beginning is not as a close a match for the PD controller as it is for the P controller. This explains the large difference in rise times. The steady state value was however an improvement upon the proportional controller. The model for the PD controller did not meet R2.3 due to the overshoot and rise time parameters.
These results demonstrate that the model for the proportional controller is extremely accurate and can therefore be used to tune for gain values. Whereas the PD controller needs further refinement.

6.1.2 Data Acquisition and Visualisation

R3.2 - The data from previous flights (plots and performance metrics) should be retained up until the program is reset.
R3.5 - Plots from previous flights should be able to be viewed on the same screen.

The proof of requirements 3.2 and 3.5 can be found in the figure below (Figure 5). The different tabs created for each flight can be seen at the top, while one of the many available views has been added through the interface (labelled Grid Container) to allow for comparison of different controllers.

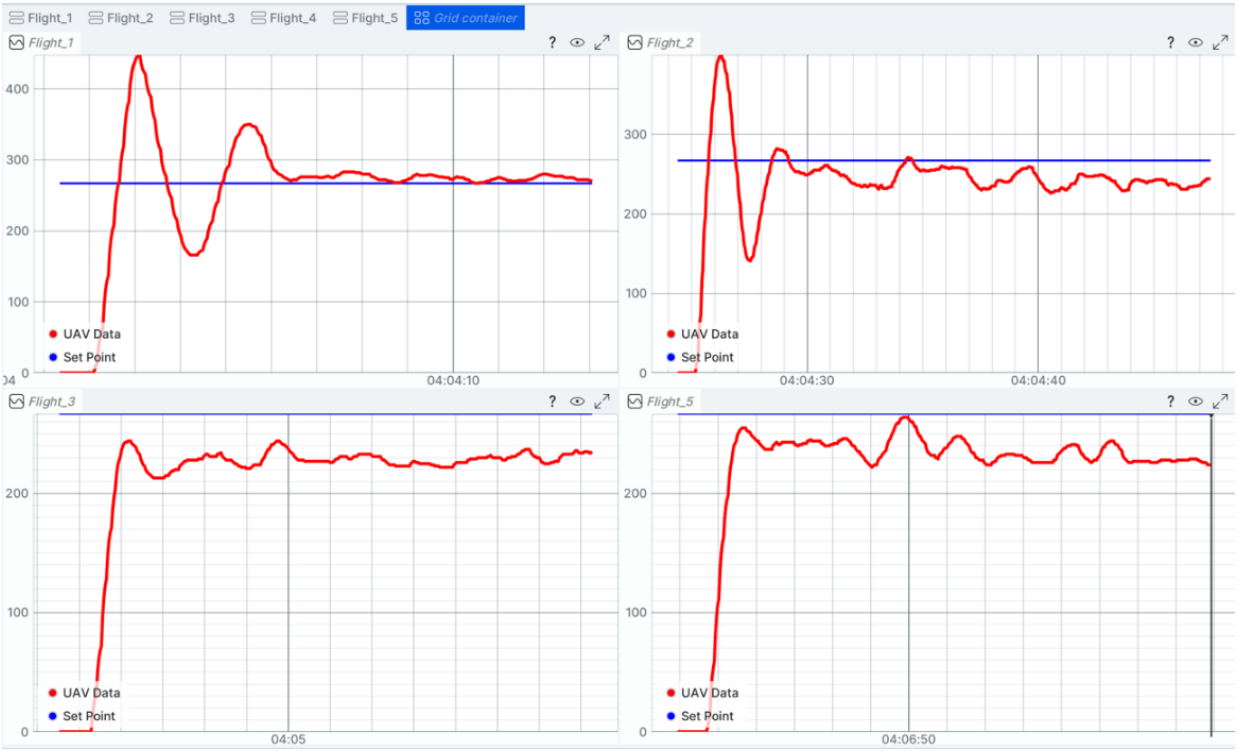


Figure 5: Grid view of multiple flight plots.

6.1.3 Performance Metrics

R4.1 - Compute and displays rise time, steady state value and overshoot where applicable for each flight.
R4.2 - Compute and displays peak value, peak time and settling time where applicable for each flight.
R4.3 - The calculated steady state value is within ±5mm of the average of the visually determined steady state window.

Both requirements 4.1 and 4.2 were completed, with a summary text file being generated for every flight including:

- Rise time
- Peak time
- Peak value
- Steady state value

- Overshoot
- Settling Time

These parameters are stored in a text file that is shown underneath the plot for each flight as seen in Figure 6.

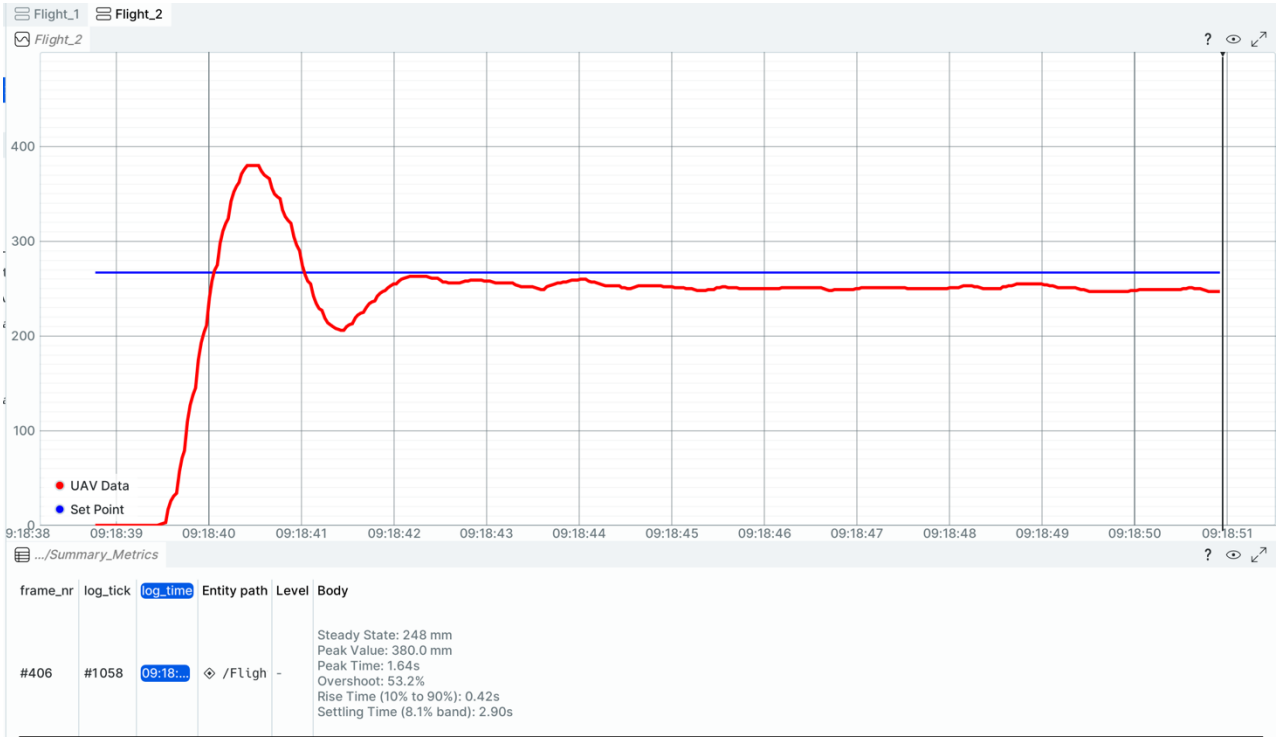


Figure 6: Standard view for each flight, plot on top and summary on the bottom.

To validate the algorithmic steady-state calculation, the height–time graph was visually inspected to determine the point at which the response appeared to settle (no further significant variation over time). A time window covering this steady region was selected, and the average height over that interval was computed. This visually determined average was then compared to the algorithmically calculated steady-state value to assess accuracy.
This was done for four sets of data including two P controllers and two PD controllers as seen in Table 6.

Table 6: Steady state value comparison

Steady State Value	P1	P2	PD1	PD2
Calculated (mm)	248	252	252	266
Visual (mm)	258	259	262	261
Difference	10	7	10	5

These results demonstrate a reasonable level of accuracy in the steady state calculation. However many of these are on the border of the requirement for accuracy with difference of ±10mm, this indicates that the window size for

calculating steady state needs to be wider to account for the variation in the data at steady state. This analysis is however limited, as visual detection of steady state can be subjective.

6.2 Limitations

6.2.1 Control Model

- The developed model assumes linear behaviour, nonlinearities are not considered.
- Only two disturbances were taken into account, other disturbances present such as the friction from the poles, are not modelled.
- The system was only analysed along the vertical axis, any effects from roll, pitch or yaw are excluded.

6.2.2 Data Analysis

- The majority of the calculated graph metrics rely on the steady state value, if this calculation is incorrect the values will be incorrect.
- The plotting system operates at 40 Hz, however latency arises from the combined delays in the Arduino IDE's serial printing, data transmission to Python, and real-time plotting via Rerun. These small delays accumulate over time, resulting in a noticeable visual lag after approximately 10 seconds. While this does not significantly impact observation, it introduces timing inaccuracies that may affect the precision of calculated performance metrics such as rise time and settling time.
- The reliance on an external program (Rerun) is a liability as the plotting relies entirely on the continued access to this free program.

7. Remaining Tasks

With the developments completed during this project, the PID-controlled flight rig is very close to being ready for use in ENEL321. There are only a few minor improvements to finalise the system.

- **PID controller tuning:** Utilise the mathematical model developed during this project to tune for a PID controller, something that isn't currently tuned on the prototype.
- **Wireless communication:** Implement wireless communication to reduce the number of cables needed during setup.
- **User Interface and Accessibility:** Develop an accessible executable that integrates the Python control script and Rerun visualisation library, enabling use across different operating systems without requiring manual setup.
- **Post Demonstration Analysis:** Implement the additional code required to allow the option for live graph replay after the demonstration.
- I**mproved Timing and Synchronisation:** Address the minor latency observed in data plotting and serial communication to ensure accurate time-based calculations and performance analysis.

- **Documentation and Teaching Resources:** Produce clear setup instructions and example experiments to support its use as a teaching tool in ENEL321.

8. Sustainability

This system promotes sustainability through its flexibility, longevity, and efficiency. By incorporating a user interface with adjustable gain values, the same hardware can demonstrate multiple controller configurations and tuning methods. This reduces the need for separate rigs, minimising material use, manufacturing demand, and long-term waste. The inclusion of real-time mapping and automated performance summaries further supports sustainable teaching practice by saving preparation time and enabling data to be reused for lectures, tutorials, and assessments.

In addition, the system has been developed with an emphasis on reliability and low maintenance, ensuring it can be used for many years with minimal intervention. This long-term usability not only conserves resources but also aligns with Engineering New Zealand's principles of sustainable design and education [6].

9. Conclusion

This project has successfully achieved all essential requirements and majority of the desirable ones. The noise analysis conducted on the system demonstrated that system noise was negligible, while disturbances had a significant impact on system behaviour. This motivated the inclusion of two key disturbances- spring effect of the cable and ground effect- in the control model to ensure accurate representation of the real system.

Using this model, the proportional and proportional-derivative controllers were simulated and compared with experimental data. The proportional model in particular, showed high accuracy when compared against experimental data, with the rise time and steady state value being over a 90% match.

The live data acquisition and plotting system successfully provided near real-time visualisation of the platform's height, and enabled calculation of key performance metrics. Such as rise time, overshoot, and steady-state error, allowing both analysis and demonstration of control concepts.

Overall this project delivered a functional and validated control model that will be utilised to tune gain values. An intuitive graphical interface that allows for near live plotting and retention of past flight data. As well as key metric calculation data for post demonstration analysis. With these additions, the flight rig bridges the gap between theory and practical that currently exists in the ENEL321 course. With the implementation of the next steps, this rig will be ready to be used for PID demonstrations in 2026.

10. References

[1] C. Bergquist, “How to Build a Drone | A DIY Guide,” *Dojo for Drones*, Mar. 23, 2019. [Online]. Available: https://dojofordrones.com/build-a-drone/

[2] **Project Code Repository: ENEL300 Project 2, hosted at University of Canterbury GitLab:**
https://eng-git.canterbury.ac.nz/cdu64/enel300_project_2

[3] “pySerial — pySerial 3.4 documentation.”
https://pyserial.readthedocs.io/en/latest/pyserial.html

[4] Rerun-Io, “GitHub - rerun-io/rerun: Visualize streams of multimodal data. Free, fast, easy to use, and simple to integrate. Built in Rust.,” *GitHub*.
https://github.com/rerun-io/rerun

[5] “Numpy.argMax — NUMPY v2.3 Manual.”
https://numpy.org/doc/2.3/reference/generated/numpy.argmax.html

[6] “Incorporate sustainable design,” *Engineering NZ*.
https://www.engineeringnz.org/programmes/engineering-climate-action/nine-climate-actions/incorporate-sustainable-design/

11. Appendix A – Control Calculations

Full derivation for thrust coefficient:

Full equation of motion:

$$m\ddot{z} = -mg + F_{Thrust} + d(t)$$

The general thrust formula:

$$F_{Thrust} = m\beta u$$

Insert thrust, remove disturbances, and divide by m:

$$\ddot{z} = -g + \beta u$$

When at hover, acceleration is zero ($\ddot{z} = 0$):

$$0 = -g + \beta u_0$$

Rearrange to find Thrust coefficient (β):

$$\beta = g/u$$

Transfer functions for the proportional-derivative controller:

Equation 11: Cable disturbance TF

$$G_r = \frac{\beta K_p + \beta K_d}{s^2 + (C_b + \beta K_d)s + \beta K_p}$$

Equation 12: Ground effect TF

$$G_d = \frac{d}{s^2 + (C_b + \beta K_d)s + \beta K_p}$$

12. Appendix C – Evaluation of Requirements

No.	Requirement	Acceptance Criteria	Rationale	Priority
1	**Control Model**			
1.1	Derive an equation of motion of the system.	Yes	Provides a baseline for the system identification.	Essential
1.2	At least one major disturbance that significantly affects system behaviour must be included in the control model.	Yes	If disturbances are playing a major role in the system, than any model excluding them will not be an accurate representation for tuning purposes.	Essential
1.3	Simulated response is reasonably accurate to the experimental response.	Yes for the proportional model, the PD model requires further work.	An accurate model is essential for tuning gains for the system.	Desirable
2	**Data Acquisition and Visualisation**			
2.1	A live viewer must plot the height measurement data at a frequency of ≥ 20 Hz.	Yes – Runs at 40Hz	This is the minimum update frequency that resembles live plotting, any less will have too much of a delay.	Essential
2.2	The data from previous flights (plots and performance metrics) should be retained up until the program is reset.	Yes – Photo Evidence	Allows for analysis post demonstration.	Desirable
2.3	Previous flight plots can be replayed post demonstration.	No – Requires additional code	Allows for analysis post demonstration.	Optional
2.4	The user can stop recording data through either a hardware or software interrupt.	Yes – Using the red button	To prevent the landing and rest data disrupting the metric calculations and graphical plots.	Essential
2.5	Plots from previous flights should be able to be viewed on the same screen.	Yes - Picture Evidence	To allow for comparison between different gains and controllers.	Optional
2.6	Provide a simple user interface to run the graphing program.	No – Currently accessed through the terminal	Improves user experience, makes it accessible to all audiences.	Optional

2.7	The plotting system should allow the appearance of the graph to be adjusted in code.	Yes – The line width and colour has been adjusted in code.	To ensure visibility in class demonstrations.	Desirable
2.8	The plotting system must integrate easily with Python. There should be valid documentation surrounding the plotting system and its integration with python.	Yes – See reference [3]	Python has access to a number of libraries making it ideal for this purpose. The main library being Pyserial to allow for communication between Arduino IDE and python.	Essential
3	**Performance Metrics**			
3.1	Compute and displays rise time, steady state value and overshoot where applicable for each flight.	Yes - Picture Evidence	Provides further information for analysis post demonstration.	Essential
3.2	Compute and displays peak value, peak time and settling time where applicable for each flight.	Yes - Picture Evidence	Provides further information for analysis post demonstration.	Optional
3.3	The calculated steady state value is within ±10mm of the average of the visually determined steady state window.	Yes	The majority of the parameters depend on the calculation of the steady state value, the accuracy of this determines the accuracy of the rise time, overshoot, and settling time.	Essential
4	The live graph program and associated scripts are easily accessible on other computers.	No – Future work	Simplifies the setup required to perform demonstrations.	Optional